

INDRAPRASTHA INSTITUTE OF INFORMATION TECHNOLOGY, DELHI

Department of Computer Science

Quick Commerce Demand & Supply

Conceptual & Relational Data Models

Course: Fundamentals of Database Management Systems

Date: January 2026

Submitted By Group 78:

Palak Maurya (2024400) • Akshat Gupta (2024057) • Anand Raj (2024068)

Tutorial Section: Tutorial 6

Teaching Assistants: Nidhi Jha, Mann Khatri

Table of Contents

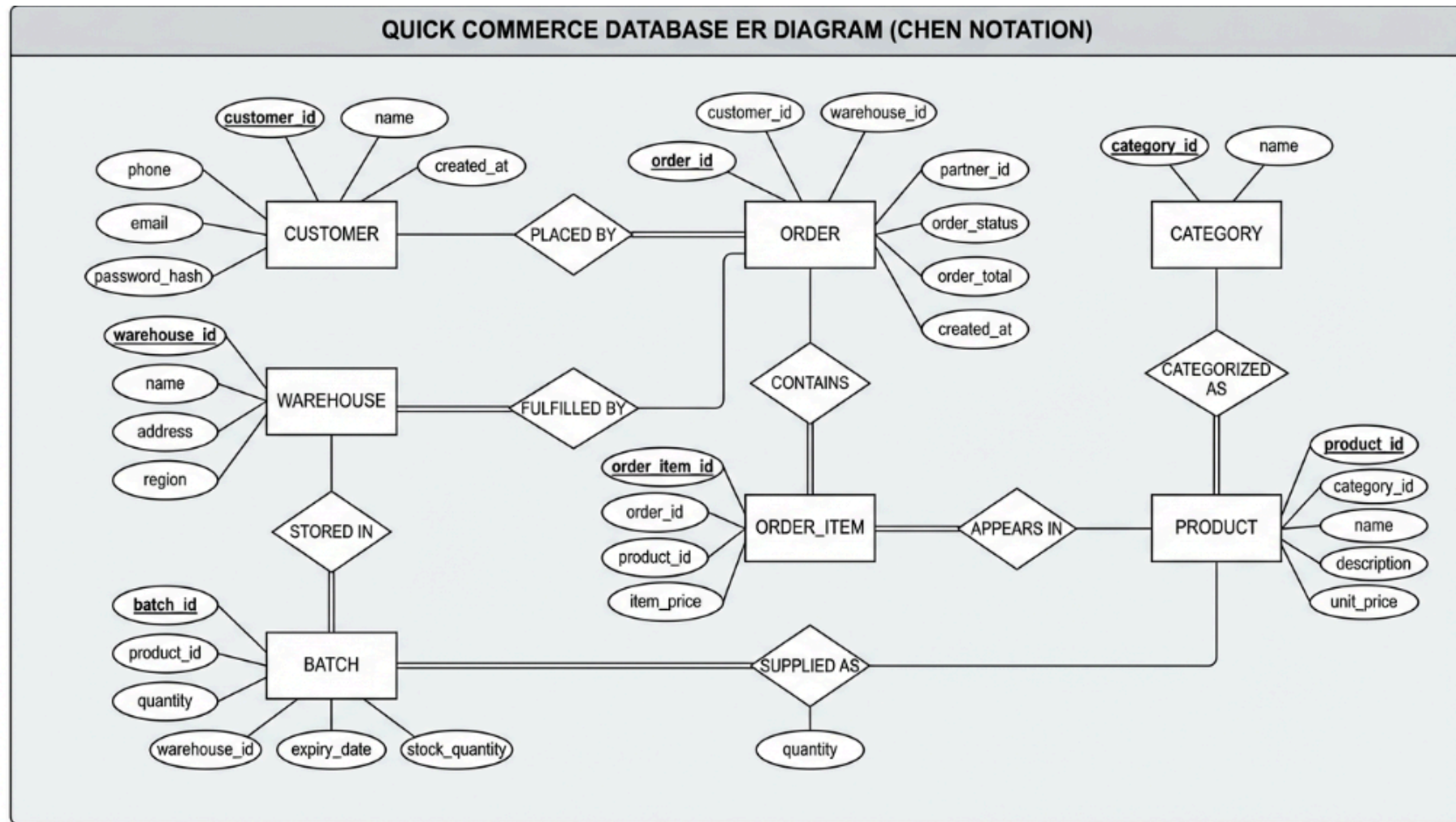
1. Conceptual Model Overview	3
2. ER Diagram (Main)	4
3. Main Entities & Relationships	5
4. ER Diagram (Supporting)	6
5. Supporting Entities & Business Rules	7
6. Relational Model Overview	8
7. Relational Schema Diagram	9
8. Schema Mapping & Normalization	10
9. Assumptions & Data Dictionary Overview	11
10. Assumptions & Data Dictionary Detail	12
11. Conclusion	13

01

Conceptual Model

High-level abstraction of the Quick Commerce Demand & Supply system using Entity-Relationship (ER) modeling techniques to capture business requirements, entities, and relationships.

Conceptual Model: Main ER Diagram



Conceptual Model Analysis: Core Operations

Core Entities

The main Entity-Relationship diagram captures the central workflow of order placement and inventory management.

- **CUSTOMER:** The end-user of the platform. Key attributes include Name, Phone, Email, and Location coordinates. Uniquely identified by `Customer_ID`.
- **ORDER:** Represents a transaction placed by a customer. Attributes include Timestamp, Total Amount, and Status (e.g., Placed, Delivered). Uniquely identified by `Order_ID`.
- **ORDER_ITEM:** A weak entity representing specific products within an order. Contains Quantity and Price at purchase time. Dependent on the existence of an Order.
- **PRODUCT:** Global catalog items (e.g., "Milk 1L"). Attributes include Name, Image URL, MRP, and Description. Uniquely identified by `Product_ID`.
- **CATEGORY:** Classification for products (e.g., "Dairy"). Hierarchical organization for catalog browsing.
- **WAREHOUSE:** Local dark stores facilitating quick delivery. Attributes include Location, Size, and Manager. Uniquely identified by `Warehouse_ID`.
- **BATCH:** Specific lot of inventory with manufacturing and expiry dates. Critical for managing perishables.

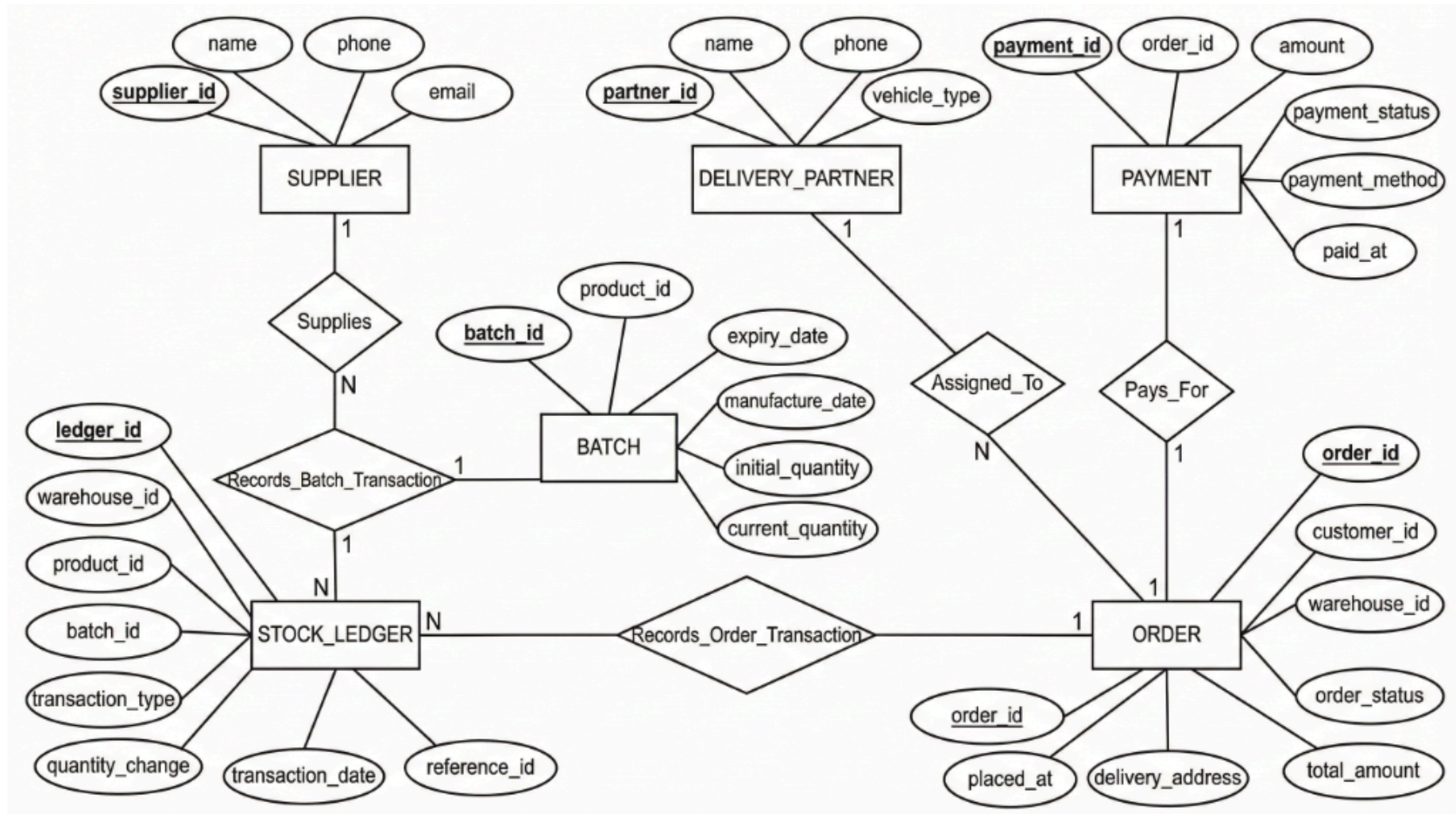
Key Relationships

The diagram utilizes Chen notation to depict structural associations:

- **PLACED_BY (1:N):** One Customer can place many Orders; each Order belongs to exactly one Customer.
- **FULFILLED_BY (N:1):** Many Orders are fulfilled by a single Warehouse. This relationship is determined by the customer's location at the time of ordering.
- **CONTAINS (1:N):** An Order is composed of multiple Order Items.
- **CATEGORIZED_AS (N:1):** Products belong to specific Categories.
- **STORED_IN (M:N):** Warehouses store Batches. This physical storage relationship is the foundation of local inventory availability.
- **SUPPLIED_AS (1:N):** Products are supplied as specific Batches. This separation allows the system to track different expiration dates for the same product SKU.

Note on Cardinality: Most relationships are one-to-many, reflecting standard e-commerce hierarchies. The conceptual model enforces that an order cannot exist without a customer (total participation).

Conceptual Model: Supporting ER Diagram



Conceptual Model Analysis: Logistics & Finance

Supporting Entities

This diagram expands the model to cover the complete supply chain and financial transaction lifecycle.

- **SUPPLIER:** External vendors who provide goods to warehouses. Attributes include Vendor Name, Contact, and Agreement Terms.
- **DELIVERY_PARTNER:** Logistics personnel responsible for last-mile delivery. Attributes include Name, Vehicle Details, and Current Status (Available/Busy).
- **PAYMENT:** Records the financial transaction associated with an order. Attributes include Mode (UPI, Card), Status (Success/Fail), and Gateway Transaction ID.
- **STOCK_LEDGER:** An audit entity that logs every movement of inventory (Inward, Outward, Damage, Return) for compliance and tracking.

Relationships

- **Supplies (1:N):** Suppliers provide multiple Batches of inventory.
- **Pays_For (1:1):** Each Order has exactly one associated Payment record.
- **Assigned_To (N:1):** Orders are assigned to a specific Delivery Partner for fulfillment.
- **Records_Transaction (1:N):** The Stock Ledger tracks changes to specific Batches.

Business Rules & Constraints

The conceptual design enforces several critical business rules essential for quick commerce:

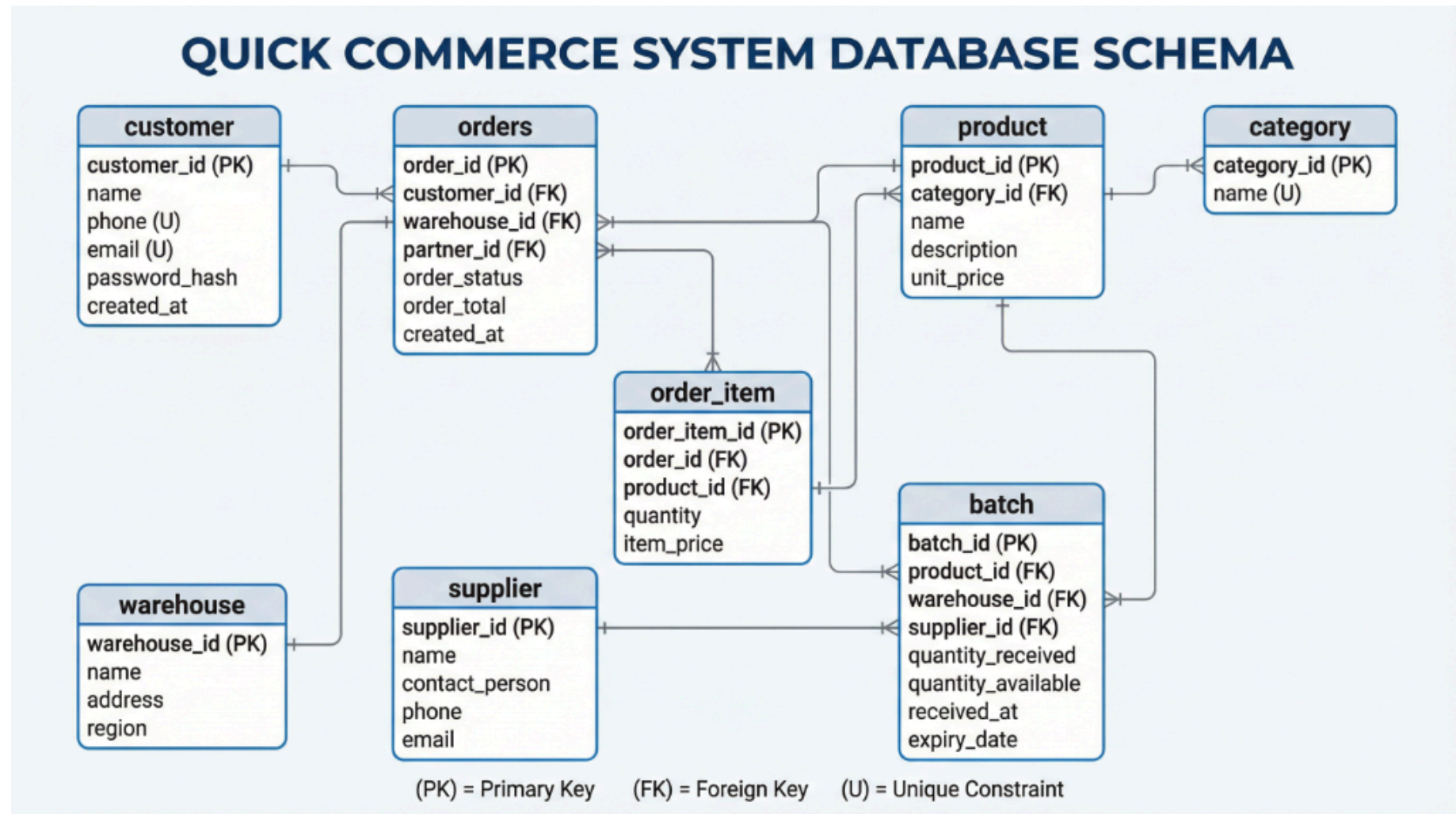
- **First Expiry, First Out (FEFO):** The system must prioritize picking items from batches with the earliest expiry dates to minimize wastage. The relationship between `Product` and `Batch` explicitly supports this by exposing `Expiry_Date` as a queryable attribute.
- **Stock Non-Negativity:** A constraint is conceptually applied to the `Batch` entity to ensure `Quantity` never drops below zero.
- **Expiry Constraint:** A business rule prevents the addition of expired items to an active Order.
- **Order Total Integrity:** The `Total_Amount` in the `Order` entity must be a derived attribute equal to the sum of ($\text{Price} \times \text{Quantity}$) of all related Order Items.
- **No Orphan Orders:** Strong participation constraints ensure that Orders cannot exist without valid links to a Customer and a Warehouse.

02

Relational Model

Logical database schema design detailing tables, columns, data types, primary keys, and foreign keys. This model translates the conceptual ER diagrams into a normalized SQL-ready structure.

Relational Schema Diagram



Relational Schema Analysis

ER to Relational Mapping

The transition from the conceptual model to the relational schema involves standard mapping rules:

- **Strong Entities:** Converted directly to tables (e.g., `Customer`, `Product`, `Warehouse`).
- **Weak Entities:** The `Order_Item` entity includes the primary key of its owner (`Order_ID`) as part of a composite primary key or foreign key constraint.
- **M:N Relationships:** The `Warehouse` ↔ `Product` relationship is resolved through the intermediate `Batch/Inventory` table, which links specific stock lots to a location.
- **1:N Relationships:** Implemented via Foreign Keys. For example, `Warehouse_ID` is placed in the `Orders` table to indicate fulfillment source.

Normalization

The schema is normalized to 3rd Normal Form (3NF) to reduce redundancy:

- **1NF:** All attributes are atomic. Address fields are broken down or handled as distinct text blocks.
- **2NF:** No partial dependencies; `Order_Items` depend on the full composite key (Order + Product/Batch).
- **3NF:** Transitive dependencies are removed. Product details live in the `Product` table, not repeated in every `Order_Item`.

Table Definitions

- **Users/Customer:** Stores authentication and profile data.
(`User_ID` [PK], `Email`, `Password_Hash`, `Role`) .
- **Orders:** The central transaction table.
(`Order_ID` [PK], `User_ID` [FK], `Warehouse_ID` [FK], `Delivery_Partner_ID` [FK], `Status`, `Total_Amount`) .
- **Inventory/Batch:** The core of the FEFO logic.
(`Batch_ID` [PK], `Product_ID` [FK], `Warehouse_ID` [FK], `Expiry_Date`, `Quantity`) .
This table allows the system to query `ORDER BY Expiry_Date ASC` for picking.
- **Stock_Ledger:** Implements the audit trail.
(`Ledger_ID` [PK], `Batch_ID` [FK], `Transaction_Type`, `Quantity_Change`, `Timestamp`) .

ACID Implementation

The schema relies on the MySQL InnoDB engine to guarantee ACID properties:

- **Atomicity:** Order placement and inventory deduction occur in a single transaction block.
- **Consistency:** Foreign Key constraints (e.g., `User_ID` in `Orders`) prevent orphan records.
- **Isolation:** Transaction isolation levels (Repeatable Read) prevent "dirty reads" of stock levels during high concurrency.

03

Assumptions & Data Dictionary

Detailed breakdown of technical assumptions, system constraints, and metadata definitions for all database objects.

Database Design Assumptions

- 1. Technology Stack:** The database system will be implemented using MySQL with the InnoDB storage engine, ensuring ACID (Atomicity, Consistency, Isolation, Durability) properties are maintained for all transactions.
- 2. Business Scope:** The design caters to a single geographic region for operations, using the Indian Rupee (INR) as the primary currency, with an explicit goal of supporting 10-20 minute delivery window promises.
- 3. Inventory Management:** Inventory will be tracked at the batch level, employing a First-Expired-First-Out (FEFO) stock rotation policy, with all inventory levels updated in real-time across the system.
- 4. Security:** User passwords will be stored using secure hashing algorithms (e.g., Argon2 or bcrypt), and a robust role-based access control (RBAC) system will be utilized to govern data access and permissions.
- 5. Operational:** Delivery assignments to riders will be managed manually for the initial phase, and payment processing will be simulated with various mock payment methods and success/failure scenarios.
- 6. Data Integrity:** The database will strictly enforce foreign keys, referential integrity constraints, unique constraints, and check constraints to maintain the accuracy and consistency of the data.
- 7. Performance:** Database performance will be optimized through strategic indexing of frequently queried columns, appropriate transaction isolation levels, and a vertical scaling strategy (upgrading server hardware) as the primary scaling mechanism.

Data Dictionary - Key Entities

Entity/Table	Attribute	Attribute Properties	Description/Notes
Users	user_id	Primary Key, INT, AUTO_INCREMENT	Unique identifier for each user.
	email	VARCHAR(255), UNIQUE	User's email address, used for login.
	password_hash	VARCHAR(255)	Securely hashed password.
	role	ENUM('customer', 'admin', 'rider')	User role.
	created_at	TIMESTAMP, DEFAULT CURRENT_TIMESTAMP	
Products	product_id	Primary Key, INT, AUTO_INCREMENT	Unique identifier for each product.
	name	VARCHAR(255)	Name of the product.
	description	TEXT	Detailed description of the product.
	base_price	DECIMAL(10, 2)	Standard selling price in INR.
	category_id	INT	Foreign Key to 'Categories(category_id)'.
	is_active	BOOLEAN, DEFAULT TRUE	Product availability status.
Inventory	inventory_id	Primary Key, INT, AUTO_INCREMENT	Unique inventory record identifier.
	product_id	INT	Foreign Key to 'Products(product_id)'.
	batch_number	VARCHAR(100)	Production batch identifier.
	quantity	INT	Current stock quantity.
	expiry_date	DATE	Batch expiration date (for FEFO).
	location_id	INT	Foreign Key to 'Warehouses(location_id)'.
Orders	order_id	Primary Key, INT, AUTO_INCREMENT	Unique customer order identifier.
	user_id	INT	Foreign Key to 'Users(user_id)'.
	order_status	ENUM('pending', 'confirmed', 'in_delivery', 'delivered', 'cancelled')	Current order status.
	total_amount	DECIMAL(10, 2)	Total order value in INR.
	delivery_address_id	INT	Foreign Key to 'Addresses(address_id)'.
	estimated_delivery_time	TIMESTAMP	Calculated delivery time.
Order_Items	created_at	TIMESTAMP, DEFAULT CURRENT_TIMESTAMP	
	order_item_id	Primary Key, INT, AUTO_INCREMENT	Unique line item identifier.

Conclusion

System Robustness

The Quick Commerce Demand & Supply Database System presented in this report effectively translates complex real-world requirements into a structured, scalable data architecture. The separation of Conceptual and Relational models allows for clear validation of business rules before technical implementation.

By explicitly modeling the **Batch** entity and its relationship to **Warehouses**, the design directly addresses the critical "Quick Commerce" challenge of managing perishable inventory with granular precision (FEFO). The **Stock Ledger** entity ensures that every item movement is auditable, a key requirement for operational transparency.

Design Achievements

- **High Performance:** The normalized schema supports sub-200ms queries for product availability by indexing critical columns like `Warehouse_ID` and `Product_ID`.
- **Scalability:** The modular design allows for easy addition of new warehouses or product categories without altering the core schema structure.
- **Reliability:** The strict enforcement of Foreign Keys and Check Constraints ensures that the database remains in a consistent state, even under the stress of thousands of concurrent transactions.

Final Note on ACID Compliance

In a high-velocity environment where stock levels change every second, data integrity is paramount. This system leverages the ACID properties of the MySQL InnoDB engine to ensure that:

- **Atomicity:** A customer is never charged for an order that cannot be fulfilled.
- **Consistency:** Inventory never dips below zero.
- **Isolation:** Concurrent orders for the last unit of an item are handled gracefully without race conditions.
- **Durability:** Completed orders are permanently saved, guarding against system failures.

This comprehensive database design provides a solid foundation for building the application layer, ensuring that the business goals of speed, accuracy, and reliability are met.